

Historically, QuickBooks had been built with large teams and long cycle times. For example, in earlier years the ill-fated online banking team had been composed of fifteen engineers, seven quality assurance specialists, a product manager, and at times more than one designer. Now no team is bigger than five people. The focus of each team is iterating with customers as rapidly as possible, running experiments, and then using validated learning to make real-time investment decisions about what to work on. As a result, whereas they used to have five major “branches” of QuickBooks that merged features at the time of the launch, now there are twenty to twenty-five branches. This allows for a much larger set of experiments. Each team works on a new feature for approximately six weeks end to end, testing it with real customers throughout the process.

Although the primary changes that are required in an adaptive organization are in the mind-set of its employees, changing the culture is not sufficient. As we saw in [Chapter 9](#), lean management requires treating work as a system and then dealing with the batch size and cycle time of the whole process. Thus, to achieve lasting change, the QuickBooks team had to invest in tools and platform changes that would enable the new, faster way of working.

For example, one of the major stress points in the attempt to release an early alpha version the previous year was that QuickBooks is a mission-critical product. Many small businesses use it as their primary repository for critical financial data. The team was extremely wary of releasing a minimum viable product that had any risk of corrupting customer data. Therefore, even if they worked in smaller teams with a smaller scope, the burden of all that risk would have made it hard to work in smaller batches.

To get the batch size down, the QuickBooks team had to invest in new technology. They built a virtualization system that allowed them to run multiple versions of QuickBooks on a customer’s computer. The second version could access all the customer’s data but could not make permanent changes to it. Thus, there was no risk of the new version corrupting the customer’s data by accident.